

09 — WOS Framework Skill + CLI Konzept

Kurzkontext

Dieses Konzept definiert ein operatives Tooling-Modell für den Aufbau eines **framework-spezifischen Skills + CLI** im WOS-Kontext.

Ziel ist, wiederkehrende Engineering-Aufgaben rund um Services, Module, Frontend-Änderungen und Tests standardisiert, nachvollziehbar und automatisierbar auszuführen.

1) Zielbild

Ein **WOS Framework Toolkit** aus zwei Ebenen:

1. **wosfw CLI** (deterministische Operations)
2. **wos-framework Skill** (planende/orchestrierende Agent-Logik mit Guardrails)

Warum beide?

- CLI ist stabil, skriptbar, auditierbar.
 - Skill ist flexibel, kontextsensitiv, kann planen/fragen/orchestrieren.
 - Zusammen entsteht: „intelligente Automation mit reproduzierbarem Kern“.
-

2) Kern-Use-Cases (aus deinen Anforderungen)

Service Lifecycle

- Neuen Service anlegen (Scaffold + Proxy + Docker Compose + Basisconfig)
- Service-Settings ändern (Ports, Env, Auth, Feature-Flags, Healthchecks)
- Services listen/inspektieren (Status, Version, Exposed Endpoints, Dependencies)

Modul-/Funktionserweiterung

- Neue Module/Funktionen in Service integrieren
- HTTP-Operationen gegen Service-Module auslösen (dev/admin operations)
- Projekt-/Mandantenfunktionen in Modulen bedienen

Frontend

- Änderungen für Desktop/Mobile Client (Aurelia)
- Optionales Scaffolding für neue Views/Routes/State-Bausteine

Qualität

- Tests erzeugen/verbessern (Unit/Integration/E2E-Stub)
 - Coverage-/Regression-Fokus bei Service- und Moduländerungen
-

3) Nicht-Ziele (wichtig)

- Kein unkontrollierter Full-Autopilot in **main**
 - Kein Umgehen bestehender PR-/Review-/Release-Gates
 - Kein produktiver Infrastruktur-Drift außerhalb deklarierter Konfiguration
 - Kein verdecktes Schreiben in fremde Services ohne expliziten Scope
-

4) Architekturvorschlag

4.1 Komponenten

1. **CLI Core (wosfw)**
 - Subcommands für Service/Module/Frontend/Test
 - Dry-Run + Apply Modus
 - JSON Output für Maschinenverarbeitung
2. **Template Engine**
 - Service-, Modul-, Test-, Frontend-Templates
 - versionsgebundene Profile
3. **Config/Registry Layer**
 - zentrale Service-Registry (z. B. `wosfw/services.yaml`)
 - Mapping für Proxy/Docker/OpenShift/Aurelia-Versionen
4. **Policy/Validation Layer**
 - Preflight Checks
 - Lint/Schema/Contract-Validierung
 - Sicherheits- und Konventionsprüfungen
5. **Agent Skill (wos-framework)**
 - plant sequenzen
 - klärt fehlende Inputs
 - ruft CLI deterministisch auf
 - erzeugt PR-fähige Zusammenfassungen

4.2 Betriebsprinzip

- Skill entscheidet *was* und *in welcher Reihenfolge*.
 - CLI entscheidet *wie exakt* (deterministisch) und führt aus.
-

5) CLI Command Surface (v1 Vorschlag)

5.1 Service

- `wosfw service add <name> --type rest --auth jwt --port 80xx`
- `wosfw service set <name> --key <k> --value <v>`
- `wosfw service list [--json]`
- `wosfw service doctor <name>`

Effekt von `service add`

- Service-Basisstruktur erzeugen
- Proxy-Routing ergänzen
- Docker Compose Service hinzufügen
- `.env.example` + Healthcheck + Basistests erzeugen

5.2 Module

- `wosfw module add <service> <module-name> --route /...`
- `wosfw module call <service> <module> <function> --payload @file.json`
- `wosfw module list <service>`

5.3 Frontend (Aurelia)

- `wosfw frontend change --target desktop|mobile --feature <name>`
- `wosfw frontend scaffold view <feature> --client desktop|mobile|both`
- `wosfw frontend scaffold component <name> --client desktop|mobile|both`

- `wosfw frontend scaffold route <path> --view <view> --client desktop|mobile|both`
- `wosfw frontend list views [--client desktop|mobile|both]`
- `wosfw frontend list components [--client desktop|mobile|both]`
- `wosfw frontend doctor` (Checks: routing integrity, dead components, naming, imports)

5.4 Tests

- `wosfw test add <service> --kind unit|integration|e2e`
- `wosfw test improve <service> --focus auth|contracts|regression`
- `wosfw test matrix <service> [--json]`

5.5 Plan/Apply

- `wosfw plan <operation...>`
 - `wosfw apply <plan-file>`
-

6) Skill-Design (wos-framework)

6.1 Skill-Aufgaben

- Anforderungen in konkrete CLI-Pläne übersetzen
- fehlenden Kontext in max. 3 Fragen je Runde klären
- bei Unsicherheit “plan-only” statt “apply” wählen
- Ergebnis als PR-Checkliste + Change Summary ausgeben

6.2 Skill-Regeln

1. Keine direkten Main-Änderungen.
2. Immer Scope + Nicht-Scope explizit.
3. Immer Dry-Run bei neuen Services/Modulen.
4. Immer Tests + Doku-Update im selben Change.
5. Bei Auth/Proxy/OpenShift Änderungen: erhöhte Sicherheitsprüfung.

6.3 Skill-Outputs

- Änderungsplan (Schritte + betroffene Dateien)
 - Risikoabschnitt
 - Testplan
 - Rollback-Hinweis
-

7) Ressourcenmodell (Knowledge Pack)

7.1 Pflicht-Referenzen

- Service Blueprint: REST, JWT, Strukturkonventionen
- Build/Runtime Matrix: Aurelia, Docker, OpenShift
- Version Matrix + Compatibility Regeln
- Proxy- und Routing-Konventionen
- Teststandards (Unit/Integration/Contract)

7.2 Empfohlene Artefakte

- `blueprints/service-rest-jwt/`
- `blueprints/module/`

- blueprints/frontend-aurelia/
 - references/version-matrix.md
 - references/openshift-deploy-rules.md
 - references/testing-minimums.md
 - references/frontend-structure.md (Folder structure, routing, state boundaries, naming)
 - references/frontend-component-guidelines.md (Container vs Presentational, reuse policy, mobile/desktop split)
 - blueprints/frontend-aurelia/view/
 - blueprints/frontend-aurelia/component/
 - blueprints/frontend-aurelia/route/
-

8) Security, Governance, Quality & Doku-Betrieb

8.1 Security-by-default + Doku-Automation

Policy-Text alleine reicht nicht; zentrale Regeln müssen als technische Gates erzwungen werden.

Branch-/Pipeline-Gates (Pflicht)

- Kein Direct Push auf `main`.
- Jeder Feature-Change läuft über PR + verpflichtende Pipeline.
- Pflichtchecks:
 - Build/Lint/Test
 - Security-Checks (Dependency + Secrets)
 - Docs-Gate (bei Service-/Moduländerungen muss passende Doku mitgezogen werden)

Agent-/Hook-gestützte Doku-Pflege

- PR-Hook erkennt betroffene Bereiche (`services/**`, `modules/**`, `frontend/**`).
- Wenn Doku fehlt: `docs-required` Label + failing check.
- Optional: Auto-Draft von Doku-Updates als Vorschlag, aber kein Blind-Merge.

AGENTS.md Contract (minimal)

- Jede Verhaltensänderung: Code + Tests + Doku im selben PR.
- Bei fehlendem Kontext: Frage statt Annahme.
- Security-sensitive Changes (`auth`, `proxy`, `openshift`) brauchen erhöhte Review-Stufe.

8.2 Governance

- Alle CLI-Operationen erzeugen maschinenlesbares Log
- PR-Template Pflichtfelder: Impact, Tests, Rollback, Security
- Agentenänderungen bleiben klein und reviewbar (<400 LoC Richtwert)

8.3 Sicherheitsregeln

- Keine Secrets in Repo (nur Secret-Referenzen)
- JWT/Auth-Änderungen benötigen 2nd human review
- OpenShift-konfigurationsänderungen nur über deklarative Files

8.4 Qualitätsgates

- Preflight: Schema + Config + route collision checks
- Postflight: lint + tests + smoke checks
- Contract checks bei API/Moduländerungen
- Frontend Gates:

- Route uniqueness + lazy-load consistency
 - Component naming and folder conventions
 - Desktop/Mobile divergence check (shared core not duplicated)
 - UI regression test hook (snapshot/e2e smoke)
-

9) Rollout-Plan (iterativ)

Phase 1 — Foundation (1–2 Sprints)

- Service Registry + CLI Skeleton
- `service add/list/set` + `plan/apply`
- Skill v1 mit Fragenlogik

Phase 2 — Modul + Frontend (2–3 Sprints)

- `module add/call/list`
- `frontend scaffold/change`
- Basistest-Generator

Phase 3 — Hardening (2 Sprints)

- OpenShift/Compose Validierung
- Idempotenz + Drift-Checks
- bessere Error Taxonomy + Recovery Hints

Phase 4 — Productization

- semantische Versionierung des Toolkits
 - Team-Onboarding Pack
 - KPI-basierte Optimierung
-

10) Erfolgsmetriken (KPIs)

- Time-to-add-service (Median)
 - Time-to-first-green-PR für neue Module
 - Regression-Rate nach scaffolding-basierten Changes
 - Anteil Changes mit vollständigem Test-/Doku-Update
 - Wiederverwendungsrate von CLI/Skill statt manueller Ad-hoc-Änderung
-

11) Risiken & Mitigation

1. **Template-Veraltung**
 - Mitigation: Version Matrix + Compatibility Tests
 2. **Zu viel Automationsvertrauen**
 - Mitigation: plan-first + human gates
 3. **Service-spezifische Sonderfälle**
 - Mitigation: Extension Hooks pro Service
 4. **Komplexe Frontend-Varianten**
 - Mitigation: Desktop/Mobile Profiles + shared core patterns
-

12) Shared Services Strategie (wichtig)

Shared Services sind zentrale Querschnittsbausteine (z. B. Auth, Files, Jobs, Tile-Services), die von mehreren fachlichen Services/Modulen genutzt werden. Änderungen daran haben hohen Impact.

Klassifikation

- **Domain Service:** fachschalenspezifisch, lokalere Auswirkungen
- **Shared Service:** domänenübergreifend, hoher Kopplungs-/Regressionseinfluss

Regeln für Shared Services

1. Versionierte Contracts (API/Schema/Event) sind Pflicht.
2. Änderungen nur über kompatible Evolution (Deprecation statt Breaking-by-default).
3. Consumer-Impact-Analyse vor Merge:
 - Welche Services/Module konsumieren die Funktion?
 - Welche Tests decken die Consumer ab?
4. Erweiterte Testmatrix:
 - Service-intern
 - Consumer-Contract-Tests
 - Integrations-Smoke über repräsentative Consumer
5. Höhere Freigabeschwelle:
 - mind. 2 Reviews
 - obligatorischer Rollback-Plan

CLI/Skill-Unterstützung für Shared Services

- `wosfw service doctor <shared-service>` zeigt Consumer-Liste + mögliche Impact-Zonen.
 - `wosfw test matrix <shared-service>` erweitert auf Consumer-Kontrakte.
 - Skill erzwingt bei Shared-Service-Änderungen automatisch plan-first + explizite Impact-Sektion.
-

13) Annahmen (explizit)

- Service-Landschaft ist über eine Registry beschreibbar.
 - JWT/REST-Prinzipien sind als Framework-Baseline stabil.
 - Aurelia-/Docker-/OpenShift-Versionen werden zentral geführt.
 - Team akzeptiert PR-/Gate-Disziplin als Standard.
-

14) Iterationsentscheidungen (warum dieses Design)

Iteration 1 — Optionen verglichen

- **A:** Nur Skill (max. flexibel, aber wenig deterministisch)
- **B:** Nur CLI (deterministisch, aber wenig kontextsensitiv)
- **C:** Skill + CLI (hybrid)

Entscheidung: C, weil sie Planungsintelligenz + reproduzierbare Ausführung verbindet.

Iteration 2 — Bedienmodell

- Direkte Apply-Commands vs. Plan-First-Workflow

Entscheidung: Plan-First als Default (`plan -> Review -> apply`) für geringeres Betriebsrisiko.

Iteration 3 — Scope-Schnitt

- Alles gleichzeitig (Service+Module+Frontend+Tests) vs. gestufter Rollout

Entscheidung: gestufter Rollout mit frühem Service-MVP, danach Modul/Frontend, dann Hardening.

15) Konkrete Empfehlung

Start mit einem **MVP Toolkit**:

1. `wosfw service add|list|set + plan/apply`
2. Skill als Orchestrator mit strikter Question-First-Logik
3. Pflicht-Blueprints für REST+JWT, Compose, Proxy, Tests
4. Danach Modul-/Frontend-Automation ausbauen

So entsteht schnell Wert, ohne die Betriebsstabilität zu riskieren.